

Session Notes: Understanding Great Design, UI and UX — Mastering Product Management (MPM) Program

Session Details

Session Title: Understanding Great Design, UI and UX

Instructor(s): Shravan

Topics Covered:

- The documentation stack: BRD, PRD, user stories, FRD and feature chunks
- Primary and secondary research, and how they complement each other
- Sampling: population sets, sample sets and personas
- The craft of the user interview: boundaries, open-ended questions and listening for stories
- Bias in research: customer bias versus product manager bias
- Old problems, new form factors: adoption, inertia and subsidised habits
- Prioritisation and ROI computation: value, effort, T-shirt sizing and MVP versioning
- Vanity metrics and the So What test
- Controlled rollouts, A/B tests and limiting the blast radius
- Being data-enabled rather than data-driven
- Three modes of thinking: first-principles thinking, product thinking and design thinking
- Design as complete experience, not UI
- Why UX is critical: onboarding, moats and muscle memory
- The six components of great UX: usability, efficiency, perceived effort, credibility, delight, simplicity
- The design iceberg: conceptual, information, interaction and visual design
- Elements of visual design: colour, font, typography, graphics, style guides and copy
- Design systems and the AI-era define-design-ship workflow
- Working with designers and developers, redesigns, and deliberate friction
- AI models, verifiable work versus knowledge work, and the future of interfaces

Executive Summary

Great products are not the result of great intentions; they are the result of disciplined thinking made legible to an entire organisation. This session traces the full arc from a raw business requirement to a shipped, well-designed experience, and shows where product managers add irreplaceable value at each step.

The first arc covers documentation. A business requirement document (BRD) captures why a change matters commercially. A product requirement document (PRD) adds the discovered problem, a high-level solution, metrics, user stories, risks, goals and non-goals. A functional requirement document (FRD) converts user stories into tactical, verifiable feature chunks that developers can actually implement. These are not four separate artefacts but one continuous document whose real purpose is shared vocabulary: bringing business, design and engineering onto a single page. English itself is a

programming language; how precisely a requirement is written determines how well it gets built.

The second arc covers research and prioritisation. Primary research delivers depth by speaking directly with users; secondary research delivers volume and signals through data, tickets, reviews and reports. Sample sets must mirror the diversity of the population set, interviews must be dominated by the user's voice rather than the interviewer's, and questions must be open-ended stories rather than leading prompts. Once pain points are gathered, ROI computation (value divided by delivery effort) decides sequence, vanity metrics are filtered out with the So What test, and risky bets are shipped through controlled rollouts.

The third and central arc is design. Design is not UI; it is the complete experience, and it lives in the solution space. Great UX has six measurable components: usability, efficiency, perceived effort, credibility, delight and simplicity. Design itself is an iceberg with four layers: conceptual design, information design, interaction design and visual design, and each layer has its own artefacts, from napkin sketches to wireframes to user flows to design systems. Real products, from Ixigo and Magicbricks to Netflix, Duolingo, Superhuman, IRCTC and the AWS console, illustrate how these principles play out for different personas. The session closes with a look at AI-assisted building: models excel at verifiable work, humans must remain the drivers, and while traditional front-ends may fade into agentic harnesses, the core principle of design, capturing attention, never changes.

Detailed Topic Notes

1. The Documentation Stack: BRD, PRD, User Stories, FRD and Feature Chunks

What it is. The chain of written artefacts carrying an idea from business goal to implementable feature: BRD, PRD, user stories, FRD and feature chunks. Despite the names, these are not different documents. They are parts of one document, split apart in large organisations only because the systems grew so big that independent thinking was needed on each part.

Why it matters. The goal of any document a product manager writes is to give clarity and bring everybody onto a single page; if everybody is not on a single page, it does not matter what is written. A PRD is a love letter written to engineering, if written perfectly: when the author is absent, the team still understands exactly what was wanted, in exactly the way the author would have explained it. Names themselves do not matter; some companies call the BRD a "business commit", and the artefact is the same damn thing.

How it works.

- **BRD.** The goal of a business is to grow the business, adding to the top line (gross revenue) or bottom line (profit). Business stakeholders usually lack the language to speak with the people who will ship, so they write a one-pager stating objective and outcome. Example: "We will ship a subscription feature because auto-renewal will reduce churn by 15 percent (equivalently, grow retention by 15 percent), adding roughly \$100 million to the top line." Note what it omits: implementation, risks, assumptions. Only outcomes and why they matter.
- **PRD.** The product team treats the BRD as a hypothesis, performs discovery, and writes the PRD: objective and problem importance, a high-level solution, and user stories.
- **User story.** A concise description of functionality from the end customer's perspective: as a user, what am I doing (action) to achieve a desired result (benefit)? Example: "As a professional photographer, I want to easily upload pictures from my camera to my website so that I can quickly show them to my clients." A user story is directional, not implementable.
- **FRD.** The FRD makes the story tactical. "As a user, I should be able to upload photos to Instagram

on low internet bandwidth" means nothing to a developer. The FRD version: "As a user, I should be able to upload photos of type JPEG or SVG, under 2 MB, on 3G or 4G bandwidth, with upload latency not exceeding 300 milliseconds." Now there are concrete constraints and a verification gate. A great feature is verifiable; without stated constraints, "it does not upload on 3G" is open to interpretation.

- **Feature chunks.** Actionably written units inside the FRD, each with its own verification mechanism. The BRD and user story are easy; writing good feature chunks is where a product manager's quality of thinking shows.
- **The rest of the PRD.** Metrics, risks and assumptions, sometimes go-to-market, and goals and non-goals (example non-goal: "improve payment latency but do not break sign-up"). Regression, the expectation that shipping one thing breaks nothing else and baseline benchmarks hold, is an assumed part of the FRD; writing it explicitly is better. Non-functional requirements (accessibility, security, performance) belong in the same document.
- **Necessary and sufficient.** From set theory: a PRD must be thorough and succinct at once. Nobody reads 100 pages. Write the 40-page version, then ask whether the essence survives in 20 pages, then 10, then 2, then 1. Quality per word is the skill; shortening means preserving the meat, not removing essentials. The same applies to prompts: the longer the prompt, the more the model misses.
- **Sprints.** The documents converge in a sprint: a collection of tasks a team ships for some return. A sprint is called a sprint because, like a 100-metre dash, it has a goal at the end. Sprints are owned by a product manager, scrum master or engineering manager, tracked with Gantt charts, coordinated through stand-ups.

Examples and case studies.

- **A real edtech PRD.** A PRD written years ago at an edtech company for a live-classes product followed exactly this structure: objective; core problem statement; why it was being built; high-level solution; metrics defined before the user stories were written (average available watch time, assessment score, among others); the discovery section with core user problems; proposed user stories for the student side, tutor side and output side ("as a user, a student should be able to see the core screens where recorded content is available"); then the FRD with every feature specified in detail, down to "the logged-in student will have his avatar in the centre of the viewport; hovering over the avatar shows his name, location and an edit button to upload an image in the pre-class experience", with design screenshots embedded; then design links, tracking, edge cases, and open questions from engineers answered inline. A back-end developer asked what resources would be supported during class (answered: handled on the CMS side); another asked whether the team would continue with Agora, a video engineering service. A good PRD invites and answers exactly these questions.
- **PM versus PO versus BA.** In many organisations, business analysts or product owners write user stories, guided by the BABOK (Business Analysis Body of Knowledge). The only difference between a product owner and a product manager is discovery: the PO does everything from user stories through FRD; the PM also does discovery. In service companies and banks, discovery is removed because the client dictates what to build. In a product-led company, the product manager owns the whole shippable experience: everything from problem to outcome.
- **Consumer PM versus technical PM.** A technical product manager guards a subsystem's robustness, soundness and decoupling. Shopify illustrates it: merchants create and code their own stores, so the platform must support enormous numbers of merchants and transactions, with careful shared-versus-isolated database decisions. Different lenses watch the same problem: product managers watch ease of use and retention, business owners watch revenue and margins,

engineering watches latency, parallelisation and failure rates. A PM should become a T-shaped learner with the vocabulary to talk architecture (what can be parallelised, webhook versus API) without writing staff-engineer code.

- **English as a programming language.** Python is not just a programming language; neither is Java; even English is one. How a requirement is written decides how well it is implemented, and the same shared-vocabulary discipline applies to AI models: "this block on the screen doesn't look nice" means nothing to a model, and two collaborators must not call the same banner two different things.

***Key Insight:** The BRD, PRD, user stories and FRD are one document, not four, and its real product is shared vocabulary. A user story is directional; a feature chunk is verifiable; and the difference between the two is where a product manager's quality of thinking becomes visible.*

2. Primary and Secondary Research

What it is. Primary research means accessing the user directly, at one degree of separation, hearing the experience straight from the horse's mouth. Secondary research studies indirect sources: past data, support tickets, analytics, app reviews, sales conversations, surveys, industry reports.

Why it matters. Discovery exists to uncover insights, not to pile up data sources. Secondary research tells you what is happening; primary research tells you why. A retention dashboard signals churn; only a conversation reveals the reason.

How it works. Primary research gives depth but cannot scale and can carry bias (interviewing only three friendly users is not research); 500 interviews per feature are impossible, but 20 to 40 are very achievable. Secondary research gives volume without depth: a user receiving a 50-question survey will not complete it without a strong incentive, and incentivised answers skew positive. Because primary cannot scale, secondary must be extremely robust. The two run in parallel. Talking to salespeople is secondary research even though many companies label it primary: the salesperson speaks to customers daily and the input has real merit, but it remains second-hand, reporting what the customer told someone else rather than the customer's own story.

Examples and case studies.

- **App reviews.** To improve an existing app, read its store reviews; most dating apps carry terrible reviews, and whether each is accurate is unknowable, but the aggregate is a signal.
- **Consulting reports.** Industry reports such as RedSeer Consulting's circulate as shared secondary sources giving market-level signals.
- **B2B churn.** B2B customers begin evaluating competitors before any interview happens. The counter is stakeholder management as friendship: visit the customer every fifteen days for a cup of tea and a chat with no agenda. By the time a formal interview is scheduled, it is too late.
- **Internal products.** Products built for internal teams routinely show initial usage then silent abandonment; no internal dashboard has ever achieved robust organic adoption. Internal stakeholders dictate solutions ("build this"), push the result for a few days, realise they never needed it, and stop. The cure: infer problems from stories, and ask why the existing feature failed before building its replacement.

***Key Insight:** Secondary research supplies the what; primary research supplies the why. A signal relayed through a salesperson, however credible, is still not the customer's own story.*

3. Sampling: Population Sets, Sample Sets and Personas

What it is. The population set is the total set of users of a system. The sample set is the researched subset, as a rule of thumb about 1 percent of the population, chosen by random sampling.

Why it matters. A sample set is only valuable when it encompasses all the personas in the population set. If the population contains ten personas, the sample must contain all ten. A skewed sample produces wrong primary research, and everything built on it inherits the error.

How it works. Identify personas first, then sample across them; A/B testing tools can construct representative samples automatically. Persona fit also predicts participation: people decline interviews when the pain researched is not their pain. A refusal usually means the sample was wrongly chosen, because a person genuinely in pain always makes time to talk about it.

Examples and case studies.

- **The convenience-food persona.** The right interviewee for a food-delivery feature is the person living alone in a cosmopolitan city with no family nearby who orders from Swiggy every single day; that person talks at length, a casual user does not.
- **Research reduces risk without eliminating it.** Apple certainly ran user interviews before the Apple mouse and still had failures; Google Plus failed despite Google's research machinery. Humans are complete, complicated people, which is exactly why deep discovery takes serious time.
- **Variance as a diagnostic.** With a correctly chosen persona, value ratings converge: variance will not exceed roughly 20 percent. Someone who never travels by train will always rate a railway-booking pain 1 out of 10; wild variance means mixed personas, not a divided market.

***Key Insight:** A sample is valid only when it mirrors every persona in the population. High variance in answers is usually not market signal; it is a symptom that the wrong people were asked.*

4. The Craft of the User Interview

What it is. A structured conversation whose goal is not to ask questions but to extract a story. Set a boundary, then listen; the features are decided later from the transcript, by inference.

Why it matters. In most interviews, the user cannot articulate their problem; that is the entire point of the interview. If the user knew exactly what they wanted, no product manager would be needed. True user intricacies live in anecdotal experiences: an earthquake may kill ten thousand people, and on paper that is a number, but for one family it is everything. The anecdote, not the aggregate, is where insight hides, which is why top companies (Anthropic, Netflix) pay product managers million-dollar compensations: this judgment can change a product's direction. The average senior product manager salary in India sits around Rs 35 to 50 lakh, but individuals earn far above it; a 26-year-old principal product manager at Zepto built the Super Saver feature and earns well beyond the average. Averages are not reality; averages are what 80 percent of people get.

How it works.

- **Set the boundary first.** "The goal of this interview is your in-class experience; not the mentor groups, only in class." Without a boundary, vague prompts yield vague, drifting responses.
- **The 20 percent rule.** After recording, check the transcript: if the interviewer spoke more than 20 percent of the words (200 of 1,000), scrap the interview; it is biased and has no value. Just nudge the person, without bias, in directions worth hearing about.
- **Open-ended questions.** Closed-ended questions limit answers to yes or no and typically begin

with do, did, is, are, would. Open-ended questions begin with why, what, how. "Do you select which flight to book based on price?" is closed and directional; "How do you select which flight to book?" is open.

- **No leading questions.** "Would you click on the buy button?" and "If I sold it as a bundle, would you buy it?" package the desired action inside the question.
- **Closed questions as filters.** Asking "Are you vegetarian?" before exploring protein sources is a legitimate filter. Minimise closed questions; do not ban them.
- **Mirroring.** From *Never Split the Difference* by former FBI hostage negotiator Chris Voss: restate what the person said or did and append "why". "So you had a very bad day. Why do you think that is?" opens the heart; "I'm so sorry your manager spoke badly to you" closes the conversation. During product tests: "I noticed you just clicked on that button; could you tell me why?" Even a shallow answer ("the button was large") is information.
- **Feel the pain in the interview.** Bad feedback in a user interview is good feedback: it surfaced before shipping. Better to feel the pain there than after launch, when it costs the job. Avoid steering users toward reassurance; the point of user testing is not to make the team feel good.
- **Metrics precede interviews.** Without defining what you want to change, you cannot design an interview. Boundary conditions can be defined while still in the problem space, exactly as a shop owner can define "the shop is worthy if every product sells at a profit" before laying the first brick, or as choosing a formal suit becomes easy once colour and three conditions are fixed.

Examples and case studies.

- **The home-cooked meal inference.** Ask a person living alone, "How do you manage food?" and the story comes back: "I order from Swiggy every day, but I get stomach aches." The user never says "build home-cooked meal delivery"; the product manager infers she wants home-style food, at appropriate times, matched to her taste palate, and ships an experimental feature that may work or fail. Never ask "what do you want?"
- **The protein chocolate drill.** Exercise: a team developing a protein wafer or chocolate must write five open-ended questions assessing viability. Reference question: "How do you meet your protein needs today?" Strong submissions: "What would stop you from buying and regularly eating a protein chocolate?", "Walk me through what you typically eat and drink in a day", "Tell me about the last time you wanted a protein snack and what happened." Weak: "How big is the chocolate market in India?", which no customer can answer.
- **The music-discovery question.** For a music app: "How do you currently discover your music?" mentions no platform and no button, yet maps existing behaviour completely.

Key Insight: *The goal of a user interview is a story, not answers. Set the boundary, speak less than 20 percent of the words, convert every observation into an open "why", and remember that if the user could articulate the solution, the product manager's job would not exist.*

5. Bias in Research: The Customer's Bias Is an Asset, Yours Is a Liability

What it is. Bias is both a good and a bad thing for a product manager, depending on whose bias it is. Customer bias, the market's pre-existing beliefs, is exploitable; the product manager's own bias, especially confirmation bias about their own work, is corrosive.

Why it matters. After working hard on something, there is a temptation to steer interviews toward validation so the feedback does not hurt, converting research into theatre. Confirmation bias is a great thing on the customer's side and very wrong on yours.

How it works. Identify what customers already believe and use it deliberately: to launch new products under a trusted brand, to soften occasional failures ("they always do good work; give them grace this once"), and to lower adoption barriers. Simultaneously strip your own assumptions out of interview design.

Examples and case studies.

- **Maruti.** "A Maruti car is always good" is a customer bias; a Maruti badge on any car sells more. That is a market asset.
- **Banks.** "Money in SBI is safe because it is a government bank" persists although HDFC arguably provides better service; families who have banked with SBI for generations will not open accounts elsewhere.
- **Education brands.** Students who return for a second paid program act on accumulated positive bias; the new product inherits first customers for free. A trusted car seller launching a supercharger service enjoys the same transfer.
- **Why a teacher gets chosen.** A student picks a course from an inkling formed by public presence ("this person probably teaches well") before any direct experience. Brands collaborate with credible individuals precisely to borrow that bias.

***Key Insight:** Exploit the customer's bias; eliminate your own. If the interview is designed so the answers cannot hurt, it is not research.*

6. Old Problems, New Form Factors: Adoption and Inertia

What it is. Every successful "new" product solves a problem that already existed; only the form factor of the solution is new. Adoption lag is not evidence of a new problem; it is the cost of overcoming inertia.

Why it matters. Believing breakthrough products created their own demand leads to building solutions without problems, and without a problem there is no reason to build a solution.

How it works. Before the new form factor, users solved the problem another way, so new products face a learning curve and habitual inertia. Companies raise and burn money to buy habit formation through discounts, free tiers and subsidised first experiences; a strategic inflection point (such as the pandemic period for delivery apps) can accelerate adoption, but the demand predates it.

Examples and case studies.

- **ChatGPT versus Google.** Before GPT, deep questions were answered through Google and Wikipedia; the problem always existed, only the conversational form factor is new. ChatGPT's free tier exists to eliminate inertia and start the habit.
- **Swiggy and DoorDash.** High-quality food with convenience was always a need, previously met by walking to shops or memorising nearby restaurants. Swiggy subsidised first orders at nearly 90 percent discounts to buy the habit.
- **Uber.** Taxi hailing was a problem since time immemorial; Uber spent enormously so users would get hooked to the new form factor.
- **Flipkart's Big Billion Day.** The same subsidy philosophy applied to a new purchasing experience.
- **Excel versus Airtable.** A user who loves Excel resists Airtable regardless of quality; the learning curve is the incumbent's moat.
- **The astrology counter-example.** India's astrology industry exceeds \$50 billion, and astrology products launch without free tiers: where faith and habit already exist, monetisation needs no

subsidy. Free tiers are for habits that do not yet exist.

Key Insight: *The problem always existed; only the form factor of the solution is new. Adoption is bought, not assumed, and its price is inversely proportional to the strength of the habit already in place.*

7. Prioritisation and ROI Computation

What it is. Ranking candidate features by return on investment: ROI equals value created divided by delivery effort (developer time).

Why it matters. There can be a magnificent pain point that is very hard to ship and several modest ones that ship fast and improve customers' lives frequently. Without the ratio, teams gravitate to whichever feature is loudest.

How it works.

- **Score value.** Value is directional, not fixed, set by averaging ratings from customers and internal stakeholders; it always depends on persona (to a vegetarian, vegetarian food is decisive). Time-sensitivity folds into the score: a feature useful only for a short window gets its value rating raised by the business.
- **Estimate effort.** Ask engineering for a rough pre-sprint estimate: T-shirt sizing, sizing the shirt from outside without wearing it (XL, large, medium).
- **Divide and compare.** A feature worth 10 taking 10 units of development time has ROI 1; a feature worth 5 taking 1 unit has ROI 5. The second ships first even though the first is the deeper pain point, unless leadership issues a hard mandate.
- **Break ties toward the smaller feature.** With identical ROIs (10 over 2 versus 5 over 1), ship the smaller. A developer estimate of one unit has never in history landed exactly on one unit; the bigger the timeline, the more the spillover. A 20-unit estimate with 20 percent spillover becomes 25 units during which the customer receives nothing, and smaller items revert more cheaply. The arithmetic holds at any scale: 100 value over 20 units equals 10 over 2, and the 20-unit bet still loses.
- **Cadence.** Ship small features continuously (a user-friendly organisation consistently improving experience) while large experience-changing features cook in the background; Microsoft ships small Excel improvements continuously and a large update every six months.
- **MVP versioning.** A large user story decomposes into versioned feature chunks: "upload photos for my business to show clients" becomes V1 (JPG upload for clients with a website), V1.1 (website and app), V1.2 (any platform, including tablet). This ladder is the MVP; one story may hold three to five tasks that will not all fit in one sprint.

Examples and case studies.

- **Learning-path feature versus resubmission fix.** In an LMS, a generative "magic UI" learning path (ask to learn retention; the system assembles a personalised 20-minute path from all resources and past recordings in real time, with a daily-brief companion feature) rates 9 or 10 on value. Allowing students to edit a wrongly submitted case-study file rates around 5: useful, not path-breaking. If the first takes 10 units of development time and the second 1, ROIs are 1 and 5: the humble fix ships first while the flagship is planned in the background. Because a real-time LLM call has real cost, the flagship would carry a fee, perhaps Rs 499 or Rs 799 a month; pricing is part of feature design when marginal cost is non-zero.
- **Impact accounting.** Where does impact live? The numerator is impact (the averaged value rating,

for example 9 from ninety raters); the denominator is complexity as estimated by engineering. There is no third term.

Key Insight: ROI equals value divided by time, and when ROIs tie, ship the smaller feature: developer estimates always spill over, and the customer should never wait 25 units of time to receive nothing.

8. Vanity Metrics and the So What Test

What it is. The So What test: state the metric, append "so what?" If the consequence cannot be justified, it is a vanity metric with no meaning.

Why it matters. Wrong metrics produce wrong prioritisation and wrong self-assessment; founders lead with impressive numbers carrying no business content, and investors dismiss them.

How it works. "I have 1 million downloads." So what? Downloads can be bought with paid advertising. "I have 10 million likes on an Instagram post." So what happened because of that? Metrics that survive usually sharpen into ratios: 1 million downloads producing \$1 million in revenue yields revenue per user of \$1, which rises when the business grows and falls when it shrinks. The same test sizes process: changing a button colour will not move conversion 5 percent, so make a ticket and get it done; changing the payments flow with an expected 10 percent jump, which could flip negative, deserves structured pros, cons and metrics. If it changes something massive, it needs supervision; if not, make a ticket.

Examples and case studies. Entrepreneurs pitching "1 million downloads" to investors receive "so what?" because the number is purchasable. A business stakeholder proposing a feature should arrive with a number ("we ran a small proof of concept; this adds \$2 million to the top line"), because numbers make claims verifiable; "I feel this will increase sales" invites "by 0.5 percent? 20 percent? 25 percent?"

Key Insight: Append "so what?" to any metric; if nothing follows, nothing was measured. The depth of process a change deserves is proportional to the size of what it can move.

9. Controlled Rollouts and De-risking Solutions

What it is. Releasing a feature to a small fraction of the audience, typically 1 to 5 percent, observing data, and expanding gradually, to limit the blast radius.

Why it matters. Solutions are hypotheses. Even after strong discovery, a shipped solution is a novel, untested thing that can completely backfire. When solution ideas leak into the objective and hypothesis phases, the honest answer is not to suppress the subconscious but to treat the solution as falsifiable and test it small.

How it works. Choose a small percentage, run long enough for decision-grade data, often as a formal A/B test, then expand. Falsifiability decides the treatment: a genuinely uncertain bet gets a controlled rollout; a plain bug fix, logically safe, ships to everyone.

Examples and case studies.

- **Netflix** can never be sure a new feature will not degrade the experience at scale, so features go to roughly 1 to 5 percent with long observation windows.
- **OpenAI** rolls features out visibly unevenly: some ChatGPT features appear for one user and not another while the first days of real usage are watched.

- **Scale changes the calculus.** A product with 500 to 600 known, paying users can ship to everyone; if something breaks, users complain, it rolls back, and accumulated trust absorbs the miss. The same product opened to 50,000 members of the public requires controlled rollouts, because shipping untested at that scale destroys credibility permanently.

***Key Insight:** Every solution is a hypothesis wearing production clothes. Control the blast radius: a small slice, enough time for real data, then expand. Only logically safe fixes skip the queue.*

10. Data-Enabled, Not Data-Driven

What it is. A product manager should not be data-driven but data-enabled. Data-driven means shipping only when data exists; but data cannot exist for a product that has not shipped. Data-enabled means data is one vector among several.

Why it matters. Strictly data-driven thinking is circular for anything new. Judgment, the ability to uncover something about a product experience others do not see and to add micro experiences (a button, a click) or macro experiences (an overhaul), is what product sense actually is, and judgment integrates multiple signals.

How it works. Assemble the vectors: quantitative data, qualitative feedback, research, business know-how. Weigh them, make a concise decision, and de-risk falsifiable bets with controlled rollouts.

Examples and case studies.

- **Buying a car.** A buyer is data-enabled: mileage (20 kilometres per litre) is data; comfort is quantified through proxies like boot space; aesthetics is judgment; the final decision integrates all into a bet that might fail.
- **Competitor-customer outreach.** One participant building an AI remedy layer researched by visiting competitors' websites, identifying their showcased customers, and cold-messaging those customers on LinkedIn about their tools, experiences and pain points. A decent approach, but one vector, biased toward existing solutions: looking at a building and asking what people dislike yields improvement signals, not novel creativity. Complement it with internal data, scraping, industry research reports and reading where the market is heading.
- **Betting ahead of data.** Agentic teams, AI employees working alongside humans, lack adoption data, yet serious people describe them as where the world is moving; one single-person startup building an agentic-team product raised \$35 million at a \$300 million valuation with exactly one person in the company. Futuristic bets are made on direction and conviction, then de-risked, because waiting for data means never moving first.

***Key Insight:** Data is one vector in a decision, not the decision. Being data-driven about a product that does not yet exist is a logical impossibility.*

11. Three Modes of Thinking: First Principles, Product Thinking, Design Thinking

What it is. First-principles thinking identifies the core value in the problem space: every customer wants less cost, more value. Product thinking decides what product embodies it: to book flights, build an app. Design thinking is the art of shaping the interface so users can fully realise the product's value.

Why it matters. A feature that exists but cannot be found may as well not exist: if Uber's cab-request screen were buried three or four clicks deep, the feature would technically work and nobody would use it.

How it works. First-principles thinking lives in the problem space; product thinking straddles both spaces, converging pain points into benefits and an MVP; design thinking lives strictly in the solution space and begins only after the MVP and feature chunks are frozen. The two-diamond pipeline makes this concrete: the discovery diamond starts from a hypothesis, diverges through research and converges to pain points and a product value template (must-haves, performance benefits, delighters); the solution diamond freezes the MVP feature set; design thinking then takes over.

Examples and case studies.

- **Flight-search convergence.** Ixigo and MakeMyTrip present essentially identical search interfaces: from, to, departure, return, search. The convergence is design thinking arriving at the same answer for the same job, placed where users expect it; buried lower on the page, the feature would die regardless of functionality.
- **iPhone migration.** Design does not mean UI; it means complete experience. Migrating all data from an old iPhone to a new one takes 20 to 30 minutes and simply works, on a device that is effectively a supercomputer capable of running its own models. Design means the simplicity a new user experiences.
- **LMS layout.** Announcements sit directly below the dashboard entry because people read left to right, top to bottom; placed lower, they would be missed. Learning content runs as a horizontal sequence matching reading order; it could have been a filter, but the scan order won. If understanding an LMS takes a person two hours, that is bad design; two minutes is great design.

***Key Insight:** First principles find the value, product thinking builds the container, and design thinking lets users actually collect the value. Design begins after the MVP freezes, and its metric is time-to-value for a new user.*

12. Design Is Experience, Not UI, and It Is Always For a Persona

What it is. Great design is frequently mistaken for great UI. UI is the final layer of design and not even the most important one. Product managers do not design anything themselves, but it is their responsibility to enable the design team to build great design.

Why it matters. Judging design without knowing its persona produces confident, wrong verdicts. Design quality is always relative to the persona it serves.

How it works. Evaluate design against the intended user's goals, vocabulary and frequency of use, never personal taste. Ask "who is this for?" before "do I like it?"

Examples and case studies.

- **The ketchup bottle.** Between a classic glass ketchup bottle and a squeezable upside-down one, the classic wins on looks (the squeezable one barely resembles a ketchup bottle) while the squeezable wins on UX: more ketchup with less effort. Good UI does not automatically mean good user experience, which is why UI and UX design were, and are, separate disciplines; the National Institute of Design runs separate tracks such as industrial design and product design for this reason.
- **The AWS dashboard.** Most general users dislike the AWS console, but they are not the persona: they do not know what EC2 or Redshift is. For a developer using it daily, it is a great way of organising an enormous service surface.
- **The terminal.** The command line looks hostile to most people and was never built for them; for its audience it is a superb tool (AI tools such as Claude later arrived inside it, but its design ignores

general audiences). Swiggy could never ship a terminal-like interface; its persona would not use it.

- **Designing for elders.** A 25-year-old designing for senior citizens does not design to personal preference: larger fonts are a persona requirement, not a taste. Within persona-appropriate bounds, remaining disagreements between variants are settled by A/B tests. Design, eventually, is judgment exercised on the user's behalf.
- **One image, many readings.** The same dashboard image reads differently to a professional (attention snaps to red and amber) and to a child (attention goes elsewhere entirely). Your view is your view, not the right view; right and wrong are defined by the end user.

***Key Insight:** UI is the last 10 percent of design. The AWS console and the terminal are great designs for the people they were built for, and no product is "badly designed" until you have named who it was designed for.*

13. Why UX Is Critical: Onboarding, Moats and Muscle Memory

What it is. UX matters for three reasons: it enables a smooth transition into the product, it is a differentiating layer, and it builds muscle memory.

Why it matters. People underestimate that a short learning curve is itself a moat: if a product takes very little time to understand and use, competitors cannot easily pull its users away.

How it works. Smooth transition: buying a new iPhone imports all apps, logs the user back into WhatsApp and most accounts, and completes seamlessly. Differentiation: when two products deliver the same functionality, the one learned faster wins. Muscle memory: move Instagram's notification icon and users' thumbs land on the wrong place; move a daily-used item in any dashboard and users hunt for it in frustration. Stable layouts compound into automatic use, the deepest retention.

Examples and case studies.

- **Superhuman and sequential design.** Superhuman, an email client later acquired by Grammarly, popularised sequential design: the product tells you the next step so you never have to find it. It surfaces the most important email, you handle it, then the next, inducing a state of flow, where Gmail makes the user think (drafts? spam? where next?). Superhuman's design metric was audaciously concrete: inbox zero, meaning every piece of information processed, the irrelevant automatically discarded, the relevant surfaced. A design goal expressed as a metric marks a serious design culture.
- **Excel's hidden thousand features.** Excel contains over a thousand features; most users can name VLOOKUP, HLOOKUP and vaguely Index, without being able to explain Index. When users know 3 of 1,000 features, design is failing to expose the product's own value; Jira draws the same criticism. Good design lets users do more with the product than they expected.

***Key Insight:** A short learning curve is a moat, and muscle memory is retention encoded in the body. The highest compliment a design can earn is that users stop thinking about it.*

14. The Six Components of Great UX

What it is. Great UX decomposes into six measurable components: usability, efficiency, perceived effort, credibility, delight and simplicity.

Why it matters. Without functional definitions, design debates collapse into taste wars; with them, a

team can instrument a product and know which lens is failing.

How it works.

- **Usability.** If 100 people start a flow, what percentage finishes without assistance? Like a game: of 100 who start, how many finish?
- **Efficiency.** What percentage finishes within a target time (say, under two minutes, or whatever session-length threshold the business sets)? A product can be usable but inefficient: a user may spend two hours on the EPFO website to withdraw money and eventually succeed; the flow completed, the design failed. IRCTC ticket booking and the income-tax e-filing portal are the same story: users grind through because they must.
- **Perceived effort.** How cluttered does the page feel? With 25 faces on a video-call screen only a few can be glanced at; with four, everything is visible. Amazon and Zepto carry very high perceived effort as a deliberate trade-off (show more products versus feel calm). Netflix takes the opposite trade: one dominant banner, one decision; opening Prime Video and seeing a single He-Man and the Masters of the Universe banner led to a click and a watch. It is easy to make something complex; it takes massive design thinking to make something simple.
- **Credibility.** Social proof that reduces the fear of deciding alone: "100 people like me have already done this." Customer logos ("used by Tesla"), alumni walls and purchase counters serve this; cars are bought after other people have visibly bought the same car.
- **Delight.** A feature never expected that genuinely works. Face unlock on the iPhone was a delight at launch; today it is an expectation, the natural life cycle of delight. A vertical scrolling video feed inside a learning platform, consumable like Instagram, is delight in an LMS context.
- **Simplicity.** The number of steps to the intended goal. Three clicks to start watching a video (open the section, click watch, unmute) is countable, and every extra click adds drop-off. The rule generalises: a page seven clicks deep in a sitemap will effectively never be discovered or surfaced by Google.

Key Insight: Usability counts completions, efficiency counts time, perceived effort counts clutter, credibility counts social proof, delight counts pleasant surprises, and simplicity counts clicks. Six lenses turn "is this good design?" from an argument into a measurement.

15. The Design Iceberg: Conceptual, Information, Interaction and Visual Design

What it is. Design is an iceberg with four layers, worked strictly in order once the MVP is frozen: conceptual design, information design, interaction design and, as the visible tip (roughly the top 10 percent), visual design.

Why it matters. Teams that jump straight to visuals produce beautiful products with broken structure. Working the ladder in order lets cheap layers absorb the churn: a napkin sketch is thrown away painlessly, a finished visual design is not. The ladder transfers directly to AI-assisted building: walking an AI agent through concept, then information structure, then interactions, then visuals produces dramatically deeper output than "design something", because the agent receives constraints layer by layer instead of guessing all four at once.

How it works.

- **Layer 1: Conceptual design.** What is envisaged before anything touches paper. Say "ride-hailing product" and a mental interface appears: a map with cars on it; say "maps-like product" and a map interface is conjured automatically. The artefact is the napkin sketch, thrown away as casually as a

handkerchief; being low in the hierarchy is what makes it disposable. In-class exercise: produce the conceptual design of a product that finds the perfect flatmate. Responses converged on filters for food preferences, language, verification, pets and budget; the sharper lesson was filter quality. "Interests" is a vague filter and produces vague results; age, verification status and "do you have pets, yes or no" are objective. Vagueness is welcome in the problem space and in user research; once a product ships, the solution side must be objective. Comparing against Magicbricks' rental flow (property type, budget, locality, tenant preferences, amenities, further filters) showed how conceptual designs converge on similar structures for similar jobs.

- **Layer 2: Information design.** The structure and hierarchy of information on the canvas. Reading order is the physics: top to bottom, left to right (right to left in locales such as Iran). Amazon's banner sits at the top because it is seen first; when a new iPhone launches, its icon occupies the today's-deals position because that placement gets noticed and clicked first. The sitemap belongs to this layer: on an e-commerce site, home page, category page, product page is a three-click hierarchy, and a category that is blowing up can be promoted so a hot product page sits one click from home, routing more traffic to it. (A worked example from commerce: the trimmers-and-epilators category at Flipkart followed exactly this home-category-product hierarchy.) Sitemaps therefore matter enormously for SEO (Search Engine Optimisation) and now GEO (Generative Engine Optimisation, how ChatGPT-class systems discover a site): a well-defined sitemap surfaces more. A live example: a program's public website reads top-down as headline, alumni, meetups, instructor bio, an information-design ordering with acknowledged faults. The artefact is the wireframe: boxes only, no visual element, not clickable; historic tools such as draw.io built these, and prompting now does, but only if the prompt states that the artefact wanted is information design. The wireframe is also where product managers should surrender creativity: it tells the designer what is important and what is not, and stops there. Most PMs do the opposite, producing a finished design and saying "implement this", which wastes the designer's craft.
- **Layer 3: Interaction design.** What the product does when the user acts: payment-confirmation sounds and screens; a music player's animated wave rising when a track plays; icons fanning out on click; a loader masking a raw loading state. Artefacts: clickable wireframes and user flows (home screen with banner, arrow, product page: if the banner is clicked, this is where it leads). Exemplars: the iPhone notification sound, so distinctive that among a hundred Android phones one iPhone chime turns heads; Duolingo, whose win sound is deliberately elated while its loss sound is deliberately gentle so failure never demoralises. A counter-example: WhatsApp's file-versus-image upload renders an image sent "as file" without a preview, and users who choose wrong must delete and re-upload; a small interaction decision causing daily friction.
- **Layer 4: Visual design.** Colours, sizes, spacing, typography; where design systems are built so identical components are reused everywhere. Detailed in the next topic.

The full user experience workflow: business case, market analysis, product strategy, MVP; then user needs and site objectives; then information design; interaction design; visual design; then coding. Companies fall into five types: (1) engineering alone defines, designs and codes; (2) product defines, nobody designs, engineering picks it up; (3) product dabbles in design alongside engineering; (4) product defines and engineering owns design; (5) the ideal: product, a UX team with equal standing, and engineering in partnership. Some companies place design at the very start through UX labs, teams doing primary and secondary research purely from a design perspective, asking "what is the next interface of e-commerce?"; Flipkart ran exactly such labs.

Key Insight: *Concept, then information, then interaction, then visuals: each layer is cheaper to change than the one after it, so spend the churn where it is cheap. A wireframe that merely says "this is important, this is not" is the correct handoff from product to design, and the same ladder is the highest-leverage way to brief an AI agent.*

16. The Elements of Visual Design

What it is. Visual design consists of colour, font, typography, graphics, style guides and layout, and copy.

Why it matters. The smallest visual decisions carry the largest consequences, because every user touches them every time. Systems thinking 101: local actions have global consequences.

How it works.

- **Colour.** Contrast is non-negotiable: without enough contrast, elements are literally invisible to parts of the audience. Colourblind users cannot distinguish similar hues (red versus orange, green versus light brown, yellow versus orange) while strongly contrasting pairs stay clear. Traffic lights use three distinct colours in fixed positions (top is red) precisely so the signal survives colour-vision deficiency: contrast plus position is accessible design.
- **Font.** Legibility is the point: cursive, doctor's-prescription handwriting is unreadable; a clean grid-consistent font is function, not decoration. Steve Jobs' design sense famously traces to a calligraphy course at Reed College, one of the world's best liberal-arts colleges, and ran deep enough that when a team designed a menu whose dropdown font violated the grid, he fired the team over it. The smallest things matter the biggest.
- **Typography.** Not font size: the spacing and readability of a typeface, how cramped or breathable text feels. (Related in spirit: typo tolerance, a system understanding that a near-miss typed for "Python" meant Python.)
- **Graphics.** Imagery and iconography carry meaning faster than words; a first-class element, not garnish.
- **Style guides and responsive layout.** A style guide fixes the layout grid and responsiveness. A responsive layout re-flows as the viewport changes: shrink a dashboard to mobile width and background elements re-layer and icons migrate to the bottom; widen to tablet and desktop and it recomposes again. One design, many devices, no broken states.
- **Copy.** Text is design. A search field labelled "search everything" behaves differently in the user's mind than an empty box; a label reading "who reacted" explains itself where "reacted" alone would be ambiguous. The same small text changes the entire reading of an interface, and good copy measurably increases conversion rates.

Examples and case studies.

- **Netflix's browse screen** is dense with show imagery yet ordered and contrast-managed so clutter reads as abundance rather than chaos.
- **Enterprise software is, bluntly, bad at design.** That is why HubSpot, a comparatively well-designed CRM, wins users from Salesforce, which many simply cannot operate. A feature factory, shipping features with no design coherence, is the worst thing a product organisation can become. Honest teams self-assess the same way: even a powerful platform whose design leaves no clean space for new features needs meaningful design rework, and saying so out loud is part of the craft.

- **Why enterprise design fails despite employing UX designers:** they are not listened to. The UX designer is treated as an employee who must obey rather than an expert who must be heard. The parallel with product managers is exact: "you don't build, you don't design, you don't code, why pay you 50 lakh rupees?" Because emphasis in the right direction wins games. A cricket team's captain may be neither the best batsman nor the best bowler, and without a great captain the team never wins: Stephen Fleming, New Zealand's captain, had a modest batting record next to Martin Crowe's spectacular one, yet ranks among the best captains ever because he understood the essence of the game holistically.

***Key Insight:** Copy is design, contrast is accessibility, and a dropdown font can be a firing offence: local actions have global consequences. If your UX designers are talented and your design is still poor, the problem is that nobody listens to them.*

17. Design Systems and the AI-Era Define-Design-Ship Workflow

What it is. A design system is a catalogue of reusable components (templates, buttons, surfaces, voice) and their exact appearance and behaviour, so every new interface reuses the same components and the product stays consistent. The AI-era workflow wraps this into three stages, define, design, ship, each assisted by an agent.

Why it matters. Without a design system, two developers shipping two features produce two visual languages and the product fragments. With agents attached to every stage, a single product manager can run a loop that previously required a full team.

How it works. A working example from a live LMS: the platform's entire design system exists as a referenced artefact showing how buttons, surfaces and voice look, and any new interface is built by referencing it so the whole product stays consistent regardless of which developer ships. The workflow: define requirements (product manager plus Claude Code); design the interface (product manager plus Claude's design agent, Claude Design, referencing the design system); ship (product manager plus Claude Code as the coding agent). The historical "A-team" of product manager, visual designer and engineers compresses, for basic workflows and small features, into a product manager plus agents. This removes the floor beneath which a PM needed to queue for help; it does not abolish UX and engineering partners for serious work.

Examples and case studies.

- **Prototype-speed expectation management.** A caution from practice: when client-facing teams generate HTML designs with Claude in minutes, clients approve instantly, and the development team then backs out because the features are expensive to actually build. The rule: never agree a design with a client without consulting the developers. Design speed has outrun engineering speed, and the design handshake is now a critical PM responsibility.
- **Reduce scope, never push.** When developers say something is hard to implement, reduce scope: reduced scope always reveals loose ends that were never needed, and it builds credibility with the development team, who then help willingly when it truly matters.
- **Measure prototypes.** Hand over prototypes on a platform that measures how many people used them and how, because without numbers every design debate is just "I am better, you are better".

Key Insight: A design system turns consistency from a discipline into a default, and the define-design-ship loop with an agent at each stage turns one product manager into a functioning product team for small features. But design that outpaces engineering is a liability: never show a client what your developers have not agreed to build.

18. Working With Designers and Developers: Respect, Redesigns and Deliberate Friction

What it is. The organisational craft around design: respecting specialists, managing redesigns against metrics, knowing when friction is a feature, and staging testing correctly.

Why it matters. The circle-ranking exercise proved how non-obvious design is: shown four coloured circles of different sizes and asked to rank them by importance, participants produced completely different orders, by colour psychology (red as danger, yellow as warning, green as success, blue as neutral), by size, by position, by contrast; a dashboard professional's instinct (eyes snapping to red and amber) inverted entirely when the same image was shown to a child. There are n ways to read the same screen. That is why design is hard and why designers deserve respect.

How it works.

- **Respect the craft.** Designers and developers repeatedly ask the five whys of every element: what to show, how, when, why. Craft is a function of depth, and appreciation begins with noticing everyday design: why door knobs are shaped as they are; why a porcelain plate survives a microwave (high heat capacity) while a steel plate must never go in; different designs for different purposes. Appreciating others' work at this level is also how a product manager earns respect in return.
- **Redesigns and trunk metrics.** After a redesign, users complain; muscle memory guarantees it. Do not reverse course within two days because something dipped. Identify the macro metric the redesign serves (engagement per reel rising while direct messages temporarily lull, with combined engagement growing) and watch the trunk metrics, the company's core baseline metrics. If trunk metrics are not falling, give users time to acclimatise: they crib, then adapt. In doubt, A/B test on 5 percent and decide over time.
- **Friction can be good design.** Frictionless is not a universal goal. An admissions application form is deliberate friction that filters for quality, where competitors capture a phone number and let sales call back. Friction is good when it consciously tapers out the wrong users; the persona and product decide.
- **Feature accretion and clutter.** Products built for one use case (Spotify for music, MakeMyTrip for flights) accrete features. A flights-only user resents the hotel cross-sell, but the hypothesis is sound: a traveller needs a hotel, so hotels appear and groceries do not. The more elements on a page, the less usable it becomes; companies accept the trade-off deliberately when monetisation demands it. Dynamic surfaces (recommendation-driven carousels, mood-based songs on YouTube Music, location-contextual content) are a design-philosophy choice layered on the same architecture.
- **Engagement versus revenue personas.** When one persona drives engagement and another drives revenue, either get the high-revenue persona using the product or raise the ARPU of the engaged persona. Instagram's largest engagement farm is India, where almost nobody pays, while the US pays heavily with fewer users; Amazon accepts losses in India for market share while profiting in Western markets and Japan; ShareChat had enormous engagement, could never monetise beyond B2B deals, and declined. If leadership cannot name the priority, map opportunity areas: user-first or revenue-first, and where is the business heading?

- **Testing stages.** Usability testing is part of user testing but comes earlier: after designs exist, build a clickable prototype backed by mock APIs, endpoints returning canned values with no backend logic, so people click through and reveal whether the design works before production code is written. This happens inside a design sprint (converging on a balanced Goldilocks design, covered in a subsequent session). User acceptance testing (UAT) comes at the end, once the product is built, before beta or launch.

***Key Insight:** Everybody reads the same screen differently, which is exactly why design is a craft and not common sense. Judge redesigns by trunk metrics, not by the first two days of complaints, and remember that the right friction, placed deliberately, is design working as intended.*

19. AI Models, Verifiable Work and the Future of Interfaces

What it is. Operating principles for building with AI models: what models are good at, who must stay in charge, how to evaluate new releases, and where interfaces are heading.

Why it matters. Model releases now arrive every 15 to 20 days; in the AI age, years work in days, and something completely new can change the entire thinking paradigm in a fortnight. It is like being handed a new car every fifteen days: without sound principles, the driver gets stuck whether it is a McLaren or an Ambassador. Only fundamentals survive the churn: defining aspects, understanding edge cases, shared vocabulary with the model.

How it works.

- **Three axes for tools.** Every new AI building mechanism is judged on ease of use, speed (how quickly it generates code and artefacts), and cost (how good the output is per unit of money).
- **Models are personalities.** Think of AI models as personality types with different nuances. A newly released cheaper Opus-class model behaved like an annoying, boring colleague with a flair of brilliance: occasionally astonishing, but frequently stopping tasks midway, double-checking compulsively and wandering out of scope. Knowing a model's personality is part of using it well; evaluate new models against real workflows.
- **Verifiable work versus knowledge work.** Most AI models are good at verifiable work and poor at knowledge work. Verifiable work has proof points: is the front end fast, do interactions load, is scroll depth good? Latency can be measured. Knowledge work is novel: "redesign this website completely in a 3D isomorphic view" has nothing to verify against, so the model ships something bad. Human ingenuity remains powerful because humans generate abstract constructs where no standard exists; even human judgments of unverifiable questions disagree (ask two people whether an outfit looks good).
- **Stay the driver.** In AI-assisted coding, the model should not be the driver; the human should be. The model implements what the human specifies, never the reverse, where the model suggests and the human just says yes. Practically: do not reach for Claude or ChatGPT at the start of thinking work. Form your own constructs first, then use the model to implement them; a model told vaguely to "make a website" makes its own assumptions and choices, which will not be what was wanted.
- **Do not outsource depth.** Reading books builds the depth that models then amplify. The method is one page a day sustained for months, not a hundred pages at a stretch. People who read have far more depth than people who only consume videos.
- **Generative and agentic interfaces.** Interfaces are shifting from static to intentful: the user states an intent and the interface generates in real time. Design thinking does not become obsolete:

design is a function of attention, and attention principles are permanent. The trade-off is real: no muscle memory forms when every screen is freshly generated, so reusable patterns need static anchors. Vercel's AI SDK embodies the best of both worlds, mixing static interface components with real-time generated ones (a live temperature widget, for example). Voice interfaces carry a similar trade-off: read text sometimes beats a speaking voice for clarity and memory.

- **The era of harnesses.** Traditional front-ends, websites and apps as we know them, are plausibly dying. The prediction: long-form agentic harnesses, systems you simply talk to, which discern which work to do now, which later, and why, pull tasks from your calendar unprompted, and complete them. Google search usage is already collapsing for people who get everything conversationally. Design does not die with the front end; it morphs. UX always exists as long as humans hold devices; only its shape changes.
- **Synthetic data in design.** Synthetic data exists to improve overall UX, not design systems per se: it simulates scenarios where real data is unavailable or uncapturable. It works best where verifiable schemas exist; on completely novel tasks it diverges, so define the task precisely. Its design value is visual truthfulness: a data-centre cooling dashboard cannot be designed sensibly without data that looks real, and synthetic data supplies that glimpse; it cannot substitute for accuracy, which is a function of how well the task is defined.
- **Ignore hype cycles.** "Design is dead" and "software engineering is dead" have been declared hundreds of times in two years; such claims mostly serve the stock prices and attention of those making them, exactly like influencers promising a 500 percent stock. No discipline ever dies; it morphs into newer things. Change is the only constant, but principles are not subject to it: principles do not change, only approaches, processes and tools do, which is why a 20-year-old design book remains relevant. Attention is the core principle of design, and it is permanent.
- **Good design is easy to change.** The ETC principle: good design is only good design when it is easy to change. It applies to product design and code design alike, and it is one of the fundamental principles of well-structured systems.

***Key Insight:** Models excel where verification exists and flail where it does not, so the human must supply the constructs and stay the driver. Front-ends may dissolve into agentic harnesses, but design survives every form-factor change, because its core principle, capturing attention, does not change.*

Frameworks and Mental Models

- **The Documentation Stack (BRD to PRD to User Stories to FRD to Feature Chunks).** One continuous document: business outcomes, discovered problem, high-level solution, user stories, tactically constrained feature chunks, metrics, risks, GTM, goals and non-goals. Use for changes big enough to move a significant metric; make a ticket for trivial ones. Visual: nested sections flowing top-down from business outcome to implementation constraint, ending in verification gates.
- **User Story Formula.** As a [user], I want to [action], so that [benefit]. Anchors every feature to a customer and a benefit; never hand it to developers as-is.
- **Necessary-and-Sufficient Compression.** Write long, then repeatedly halve (40 pages, 20, 10, 2, 1), testing whether the essence survives; apply to every document and prompt.
- **The Double Diamond.** Discovery diamond: hypothesis in, diverge through research, converge to pain points and a product value template (must-haves, performance benefits, delighters). Solution diamond: converge to a frozen MVP. Design thinking begins after the second diamond closes. Visual: two diamonds joined at their points.

- **Primary versus Secondary Research.** Primary: direct, deep, unscalable, bias-prone. Secondary: indirect, voluminous, shallow, signal-generating. Secondary tells what, primary tells why; run in parallel; make secondary robust because primary cannot scale.
- **Population Set, Sample Set, 1 Percent Rule.** Randomly sample about 1 percent, covering every persona. Diagnostic: value-rating variance above roughly 20 percent means wrong persona selection.
- **The User Interview Protocol.** Set a boundary; ask open questions (why, what, how); avoid closed (do, did, is, are, would) and leading questions except as filters; speak under 20 percent of the words or scrap the interview; mirror observations by appending "why"; listen for stories, infer problems, never ask for solutions.
- **The So What Test.** Append "so what?" to any metric; no consequence means vanity. Survivors sharpen into ratios such as revenue per user.
- **ROI Computation and T-Shirt Sizing.** ROI = averaged impact rating divided by rough engineering time (T-shirt sized). Prioritise by ROI; on ties, ship the smaller feature because estimates always spill over; override only by explicit business mandate.
- **MVP Versioning (V1, V1.1, V1.2).** Decompose a large user story into progressively broader feature chunks so each sprint stays shippable.
- **Controlled Rollout / Blast Radius.** Release to 1 to 5 percent, gather decision-grade data (often A/B), expand gradually. Use for falsifiable bets; skip for logically safe fixes.
- **Data-Enabled Decision Vectors.** Integrate quantitative data, qualitative feedback, research and business know-how; data-driven is circular for new products.
- **Three Modes of Thinking.** First principles (problem space: core value), product thinking (what product delivers it), design thinking (solution space: how the interface lets users realise it). Use to diagnose which conversation a team is actually having.
- **The Six Components of Great UX.** Usability (percent completing unaided), efficiency (percent completing within target time), perceived effort (clutter), credibility (social proof), delight (unexpected value), simplicity (click count). Six measurable audit lenses.
- **The Design Iceberg.** Conceptual (napkin sketch), information (wireframes, sitemaps, reading-order hierarchy), interaction (clickable prototypes, user flows), visual (colour, typography, copy, design systems). Visual: an iceberg with visual design as the visible 10 percent tip. Use for products and for briefing AI agents layer by layer.
- **The Five-Company Design Maturity Ladder.** From engineering-does-everything up to product, empowered UX and engineering in partnership. Use to locate an organisation and choose realistic next steps.
- **Define-Design-Ship with Agents.** Define (PM plus coding agent), design against a referenced design system (PM plus design agent), ship (PM plus coding agent). For small features a PM can own end to end.
- **Trunk Metrics for Redesigns.** Hold steady while core baseline metrics hold; tolerate complaints; A/B test when in doubt; never reverse after two days of noise.
- **Verifiable Work versus Knowledge Work.** Give models tasks with proof points; reserve novel creation for human construct-building, then hand constructs to the model.
- **The ETC Principle.** Easy To Change: good design, in products and code, is only good if it is easy to change.
- **The Three Axes of AI Building Tools.** Ease of use, speed, cost; evaluate every new model release on these instead of hype.

Key Terminology

- **BRD (Business Requirement Document):** A short business-side document stating an objective and expected business outcome, without implementation detail.
- **PRD (Product Requirement Document):** The master document uniting objective, problem, solution, metrics, user stories, FRD detail, design, tracking, risks and open questions; the organisation's shared vocabulary for a change.
- **FRD (Functional Requirement Document):** The tactical portion converting user stories into constrained, verifiable specifications.
- **User story:** A concise description of functionality from the end customer's perspective: user, action, benefit.
- **Feature chunk:** An actionably written, independently verifiable unit of work inside an FRD.
- **Non-goal:** An explicit statement of what a change must not affect.
- **Regression:** The expectation that shipping something new breaks nothing existing and baseline benchmarks hold.
- **Top line / bottom line:** Gross revenue / profit after costs.
- **Churn:** Users leaving a product; **retention** is its opposite.
- **Vanity metric:** A number that fails the So What test.
- **Primary research:** Direct access to the user at one degree of separation; deep, unscalable.
- **Secondary research:** Indirect sources (data, tickets, reviews, reports, sales conversations); voluminous, shallow.
- **Population set / sample set:** The total user base / the researched roughly-1-percent subset that must mirror every persona.
- **Persona:** A distinct user type defined by role, need and context; the unit of sampling and design targeting.
- **Closed-ended question:** Answerable with yes or no; typically begins do, did, is, are, would.
- **Open-ended question:** Requires explanation; typically begins why, what, how.
- **Leading question:** Packages the desired answer inside itself, contaminating research.
- **Mirroring:** Restating a person's words or actions and appending "why".
- **Confirmation bias:** Steering evidence toward what one already believes; an asset when the customer's, a liability when the researcher's.
- **ROI computation:** Ranking features by value divided by time-to-ship.
- **T-shirt sizing:** Rough outside-in effort estimation (XL, large, medium) before formal estimation.
- **Spillover:** The systematic overrun of development estimates; grows with estimate size.
- **MVP:** The frozen, versioned set of feature chunks (V1, V1.1, V1.2) delivering core value first.
- **Sprint:** A goal-bearing collection of tasks shipped in a fixed window; tracked with **Gantt charts** and **stand-ups**.
- **Controlled rollout:** Releasing to a small audience fraction to limit the **blast radius** before expanding.
- **A/B test:** Running variants against split audiences to decide with data.
- **Whole shippable experience:** Product-manager ownership of everything from problem to outcome.
- **T-shaped learner:** Broad horizontal surface area across disciplines plus vertical depth in one.

- **Product sense:** The judgment to add micro experiences (a button, a click) or macro experiences (an overhaul) others do not see.
- **First-principles thinking / product thinking / design thinking:** Core value in the problem space / what product embodies it / how the interface lets users realise it.
- **Usability:** Percent of users completing a flow without assistance.
- **Efficiency:** Percent completing within a target time.
- **Perceived effort:** How cluttered and demanding a screen feels.
- **Credibility:** Social proof that others like the user have succeeded here.
- **Delight:** An unexpected feature or experience that genuinely works.
- **Simplicity:** The number of steps or clicks to the intended goal.
- **Conceptual design:** The envisaged idea before anything is drawn; artefact: the disposable napkin sketch.
- **Information design:** The hierarchy of information on a page or site; artefacts: wireframes and sitemaps.
- **Interaction design:** What the product does in response to user actions; artefacts: clickable prototypes and user flows.
- **Visual design:** Colour, font, typography, graphics, layout and copy; the iceberg's visible tip.
- **Wireframe:** A non-clickable, box-only layout stating what is important and where.
- **Sitemap:** The click-hierarchy of a site's pages; central to SEO and GEO discoverability.
- **User flow:** A diagram of screens and the interactions connecting them.
- **Typography:** The spacing and readability of a typeface, distinct from font choice.
- **Style guide:** Fixed rules for layout grid, components and responsiveness.
- **Copy:** The words on the interface; a first-class design element that moves conversion.
- **Design system:** The catalogued, reusable component set ensuring product-wide consistency.
- **Sequential design:** Design that tells the user the next step instead of making them find it, inducing flow.
- **Muscle memory:** Automatic, body-level familiarity with interface positions, built by layout stability.
- **SEO / GEO:** Search Engine Optimisation / Generative Engine Optimisation (discoverability by generative AI systems).
- **Usability testing:** Testing a clickable prototype, often backed by **mock APIs** (stub endpoints returning canned values), before production code exists.
- **UAT (User Acceptance Testing):** End-stage testing of the finished product before beta or launch.
- **Design sprint:** A structured process converging on a validated, balanced (Goldilocks) design; treated in a subsequent session.
- **Trunk metrics:** The company's core baseline metrics that must not fall through any change.
- **UX lab:** A team doing primary and secondary research purely from a design perspective.
- **ARPU:** Average revenue per user.
- **Verifiable work / knowledge work:** Work with checkable proof points (models excel) / novel work without verification anchors (humans essential).
- **Harness:** A long-form agentic system a user talks to, which discerns, sequences and completes work; the predicted successor to traditional front-ends.
- **Synthetic data:** Simulated data used where real data is unavailable; valuable for visually truthful

design and verifiable-schema testing.

- **ETC (Easy To Change):** The principle that good design is only good design when it is easy to change.

Action Items and Homework

- **Audit your case-study interview questions.** Check each against the open-ended, non-leading, boundary-first framework and note which were directional or leading.
- **Prepare the discovery presentation.** Four minutes, strictly; lead with insights, not inferences. Stating what is visibly in the material is an inference; an insight is something uncovered that was not obvious from the surface.
- **Practise the five-question drill.** For any product idea (the in-class exercise used a protein chocolate), write five open-ended questions assessing viability without leading the user.
- **Practise conceptual design.** Take a vague brief (in class: a product to find the perfect flatmate), sketch the concept, then compare against a live product serving the same job (Magicbricks' rental flow).
- **Walk the design iceberg with an AI agent.** Brief an agent through conceptual, information, interaction and visual design in order, and compare against a single vague "design something" prompt; the difference in depth will be clearly visible.
- **Do your own thinking before opening an AI tool.** Avoid reaching for Claude or ChatGPT at the thinking stage of current coursework; form your own constructs first and use models only to implement them.
- **Build the reading habit.** One page per day, sustained for months. Start with Don't Make Me Think by Steve Krug (about 100 pages, readable in a flight).
- **Observe design in the wild.** Use Duolingo and attend to its win and loss sounds; notice door knobs, plates, traffic lights, WhatsApp's file-versus-image upload; ask of each everyday object what it is optimising for.
- **Apply the So What test** to the metrics in your current work documents; discard or sharpen anything that fails.
- **Check your interview talk-share.** Verify you spoke under 20 percent of the words in any recorded interview; scrap and redo any where you did not.
- **Expect a prototype exercise.** Prototyping is an upcoming skill, and attempting a prototype before being taught is explicitly encouraged: making mistakes first deepens the later lesson.

Resources Mentioned in the Session

Books

- **Never Split the Difference** by Chris Voss, the former FBI hostage negotiator; one of the best books on negotiation and the source of the mirroring technique. (The author's name was garbled in speech as "Chris Swoss"; the correct attribution is Chris Voss, co-written with Tahl Raz.)
- **Don't Make Me Think** by Steve Krug; a roughly 100-page classic on design and usability. Though written over two decades ago, it remains relevant because principles do not change; only approaches, processes and tools do.
- **BABOK (Business Analysis Body of Knowledge):** The standard reference for business analysts, covering how to write user stories and related artefacts. (Rendered in speech as "Babak"; the

correct name is BABOK, published by the International Institute of Business Analysis.)

Philosophical and literary references

- **Maker's Schedule, Manager's Schedule** by Paul Graham: makers need low abstraction and concrete detail (a carpenter needs exact brick and cement counts); managers coordinate at higher abstraction (a contractor needs totals). Used to explain why developers need FRD-level specificity while product managers operate at user-story altitude.
- **Set theory's necessary-and-sufficient conditions**, applied to documents: a PRD must contain what is needed and enough of it, while staying succinct.
- **The Bhagavad-Gita tradition**, invoked for the permanence of principles: the principles taught are timeless; teachers merely repeat them.

People and public figures referenced

- **Steve Jobs**: Studied calligraphy at Reed College, one of the world's finest liberal-arts institutions, which shaped his design sense; he once fired a team because a dropdown menu's font violated the design grid.
- **Chris Voss, Steve Krug, Paul Graham**: See Books and literary references.
- **Stephen Fleming**: New Zealand cricket captain with a modest batting record who ranks among the best captains ever; the argument for why orchestrating roles deserve premium pay.
- **Martin Crowe**: New Zealand batsman with a spectacular record, contrasted with Fleming to separate individual skill from leadership value.
- **Matt Pocock**: Developer who authored the Grill Me skill for Claude, which interrogates a plan and produces a strong functional requirement document.

Products and tools referenced

- **Claude, Claude Code and Claude Design**: The agent stack in the define-design-ship workflow: coding agents for defining and shipping, a design agent working against a referenced design system.
- **ChatGPT / GPT**: Free tier as inertia-removal; controlled feature rollouts; the new form factor for the old problem of getting answers.
- **Opus-class AI models**: A newly released cheaper Opus-class model showed brilliance in flashes but stopped tasks midway, double-checked compulsively and drifted out of scope; an argument for learning each model's personality.
- **Swiggy and DoorDash**: Convenience demand always existed; Swiggy subsidised first orders at up to 90 percent off to buy habit.
- **Uber**: New form factor for the immemorial taxi problem; heavy spending to build habit.
- **Flipkart**: Big Billion Day as adoption subsidy; UX labs researching the next interface of e-commerce; the trimmers-and-epilators category illustrating home-category-product hierarchy.
- **Amazon**: High perceived effort as deliberate trade-off; banner placement as information design; losses accepted in India for market share while retail profits flow in Western markets and Japan.
- **Netflix**: Controlled rollouts on 1 to 5 percent; low perceived effort with a single dominant banner; million-dollar PM compensation.
- **Prime Video**: The single He-Man and the Masters of the Universe banner that converted a browse into a watch in one click.

- **Ixigo and MakeMyTrip:** Convergent flight-search interfaces; MakeMyTrip also illustrates feature accretion and hotel cross-sell.
- **Magicbricks:** Rental-search filters as the comparison target for the flatmate-finder exercise.
- **Superhuman:** Email client acquired by Grammarly; popularised sequential design and the inbox-zero design metric.
- **Gmail:** The contrast case that makes the user think about where to go next.
- **Duolingo:** Exemplary interaction design: elated win sounds, gentle loss sounds.
- **WhatsApp:** Deeply designed overall; the file-versus-image upload flow cited as a poor design decision causing repeated user error.
- **Instagram:** Muscle-memory example (moving the notification icon breaks thumb habits); India as its largest engagement farm with minimal payment.
- **Excel and Jira:** Enormous capability (over a thousand Excel features) hidden by poor design; users know only VLOOKUP, HLOOKUP and vaguely Index.
- **Airtable:** The superior-but-resisted alternative illustrating incumbent inertia against Excel.
- **HubSpot and Salesforce:** The comparatively well-designed CRM versus the enterprise product users struggle to operate.
- **AWS (EC2, Redshift):** The console disliked by non-developers but excellent for its true persona.
- **Shopify:** Platform product management: merchants build their own stores, demanding robustness and decoupling at scale.
- **IRCTC, the EPFO portal and the income-tax e-filing portal:** Usable but deeply inefficient flows completed only because users must.
- **Zepto:** High perceived effort by design; the Super Saver feature, built by a 26-year-old principal product manager, as evidence that averages hide the top of the market.
- **ShareChat:** Massive engagement that never monetised beyond B2B deals; the engagement-versus-revenue cautionary tale.
- **Spotify and YouTube Music:** Feature accretion and mood-based dynamic recommendation surfaces respectively.
- **Google and Google Plus:** Search as the pre-GPT answer engine now losing usage; Google Plus as a research-backed product that still failed.
- **Apple:** iPhone migration UX, Face ID as delight-turned-expectation, the Apple mouse as a researched product with failures, the distinctive iPhone notification sound.
- **Maruti, SBI and HDFC:** Customer-bias examples from cars and banking.
- **Agora:** The video engineering service referenced in the edtech PRD's open questions.
- **Vercel AI SDK:** Mixing static interface components with real-time generated ones (a live temperature widget).
- **draw.io:** Historic wireframing tool from the pre-prompting era.
- **Grill Me (Claude skill):** Matt Pocock's skill that grills a plan into a strong FRD.
- **RedSeer Consulting:** Source of industry research reports used as shared secondary research.
- **A single-person agentic-AI startup:** Raised \$35 million at a \$300 million valuation with exactly one person, the whole product being an agentic team; evidence of where building is heading.

Community and ongoing learning

- **National Institute of Design (NID):** Separate tracks (industrial design, product design) evidencing

that UI and UX are historically distinct disciplines.

- **Reed College:** Where Steve Jobs studied calligraphy.
- **A recorded business-school lecture series on depth in product analytics** was promised as follow-up material covering redesign metrics with worked examples.
- **A B2B product leadership interview** (with a director of B2B at Sabre) was referenced for relationship-first B2B stakeholder management.
- Upcoming program material: LLM theory (tokens, tokenisation, weights, attention, reasoning levels), prototyping, design sprints and the Goldilocks design, metrics design (baseline, leading and lagging metrics), the design handshake with engineering, managed agents on persistent machines, and harnesses.

Session Takeaways

1. The BRD, PRD, user stories and FRD are one document with one purpose: bringing the whole organisation onto a single page with a shared vocabulary.
2. English is a programming language: how precisely a requirement is written determines how well the feature is built, whether the builder is a developer or an AI model.
3. A user story is directional; a feature chunk is verifiable. A great feature is one you can verify, and writing verification gates is where a product manager's thinking quality shows.
4. Secondary research tells you what is happening; primary research tells you why. A salesperson's account, however close to the customer, is still not the customer's story.
5. The goal of a user interview is a story, not answers: set the boundary, speak less than 20 percent of the words, ask open-ended questions, and infer the problem the user cannot articulate.
6. Exploit the customer's bias; eliminate your own. Bad feedback in a user interview is good feedback, because it arrived before launch.
7. The problem always existed; only the form factor of the solution is new. Adoption is bought with subsidised habit, not assumed.
8. ROI equals value divided by time, and when ROIs tie, ship the smaller feature: developer estimates always spill over, and customers should never wait months to receive nothing.
9. Append "so what?" to every metric; if nothing follows, it is vanity. A million downloads bought with advertising proves nothing; revenue per user proves something.
10. Design is not UI. UI is the final 10 percent of an iceberg whose deeper layers, conceptual, information and interaction design, decide whether users ever realise the product's value.
11. Great design is measured, not felt: usability, efficiency, perceived effort, credibility, delight and simplicity are countable, and the ultimate metric is how little time a new user needs to succeed.
12. Judge every design by its persona, never by your taste: the AWS console and the terminal are great designs for the people they were built for.
13. In AI-assisted building, the human is the driver and the model is the implementer: form your own constructs first, because models excel at verifiable work and flail at novel knowledge work.
14. No discipline ever dies; it morphs. Front-ends may dissolve into agentic harnesses, but attention is the core principle of design, and principles do not change.